AMD EMBEDDED DEVELOPMENT FRAMEWORK

Migrating a Vivado design to an EDF Linux image with agentic AI

A repeatable path from an existing hardware design to a reproducible, flashable Kria image: the agent does the reading and cross-checking, the tools hold the veto, the engineer drives.

1 · Interrogate
Extract the design's real IP configuration, clocks and address map

2 · Rebuild PL
Scripted block design, validated and implemented; bitstream + XSA

3 · Firmware
Bitstream, overlay and shell.json packaged for dfx-mgr

4 · Image
Custom layer on EDF; BitBake builds and is inspected offline

5 · Bring-up
Boot and verify one boundary at a time; fixes go back into sources

Coding agent

Reads project sources, Vivado reports, AMD docs, kernel bindings and BitBake metadata; drafts Tcl, overlays and recipes; proposes the next bounded check.

Vivado MCP

Gives the agent a live Tcl session: query IP properties and address segments, run `validate_bd_design`, synthesis, implementation and reports directly.

Engineer + standard tools

Runs `repo`, `bitbake`, `dtc`, `dd`, `journalctl` and the board; approves every change and owns decisions.

STEPS 1–2 · HARDWARE

Establish what the design actually is, then rebuild it for Linux

1. Interrogate the existing project via Vivado MCP

- Agent opens the project read-only and extracts IP configuration, interface widths, clock domains, interrupts and the full address map.
- Record the result as a written hardware contract. Trust the artifact, not the documentation: configured link speeds, widths and offsets are frequently not what the requirements said.
- Identify what cannot carry over: locked or missing local IP, bare-metal-only register conventions, fixed physical buffers.

```
report_property [get_bd_cells -hier *]
get_bd_addr_segs -hier ; report_ip_status
get_property CONFIG.* [get_bd_cells dma_0]
```

2. Rebuild the PL as a script, not a patched GUI project

- Agent writes a Tcl build for the Linux variant; leave the original project untouched as reference.
- Prefer IP with upstream Linux drivers (e.g. framebuffer/DMA IP with V4L2 or ALSA support) over custom registers driven from userspace.
- Iterate on `validate_bd_design` through MCP: unused PS ports, mismatched AXI4-Stream widths, unnecessary clock crossings. Fix the cause; don't silence the check.
- Check the clock arithmetic by hand before accepting it.

```
launch_runs impl_1 -to_step write_bitstream ; wait_on_run impl_1
report_timing_summary ; report_drc
write_hw_platform -fixed -include_bit design.xsa
```

Gate: positive setup and hold slack, zero DRC errors, a bitstream and fixed XSA whose address map matches the written contract.

STEP 3 · FIRMWARE BUNDLE

Describe the PL to Linux: overlay, shell.json, firmware recipe

Device-tree overlay

- Agent drafts the `.dtso` from three sources: the Vivado address map, AMD reference overlays, and the kernel's DT bindings for each driver.
- Cross-check each node: `reg` against address segments, `interrupts` against the PL-PS IRQ concat, clocks, I²C muxes, regulators and media/sound graph endpoints.
- Compile early to catch syntax and phandle errors before BitBake does.

```
dtc -@ -I dts -O dtb -o app.dtbo app.dtso
fdtdump app.dtbo | less
```

Firmware recipe (in your own layer)

- `shell.json` declares a flat PL design for `dfx-mgr`.
- `DEPENDS = "dtc-native"` is a build-graph edge; `RDEPENDS:${PN} = "dfx-mgr"` is a runtime edge on the target.
- Install into `${D}/lib/firmware/xilinx/<name>`; `${D}` is a staging root, not the board.

```
SRC_URI = "file://app.bit.bin file://app.dtso \
           file://shell.json"
FW = "${nonarch_base_libdir}/firmware/xilinx/app"
FILES:${PN} = "${FW}"
do_compile() {
  dtc -@ -I dts -O dtb -o app.dtbo ${S}/app.dtso
}
do_install() {
  install -d ${D}${FW}
  install -m 0644 ${S}/app.bit.bin ${B}/app.dtbo \
    ${S}/shell.json ${D}${FW}/
}
```

STEP 4 · IMAGE

Build on EDF; inspect the image before flashing

Workspace

- AMD EDF build container; bind-mount a host directory so sources and caches persist.
- Rootless Podman: use `--userns=keep-id`.
- Source `edf-init-build-env` from Bash.

```
repo init -m default-edf.xml \
  -u github.com/Xilinx/yocto-manifests \
  -b refs/tags/amd-edf-rel-v25.11
repo sync
bitbake-layers add-layer ../meta-app
```

Layer contents

- Image recipe: EDF Kria base + application, firmware, tools.
- Application recipe and systemd units: wait for PL load, then start.
- Kernel options as a `.cfg` fragment via `linux-xlnx_%.bbappend`; verify in the generated `.config`.
- Agent typechecks `MACHINE`: one selected name → `conf/machine/*.conf` → `MACHINEOVERRIDES` → `COMPATIBLE_MACHINE`. Keep all recipes in the same domain.

Build and inspect

```
export MACHINE=kria-zynqmp-generic
bitbake app-image
bitbake -e app-image \
  | grep ^IMAGE_INSTALL=
```

- Agent checks the manifest, firmware package contents and default-firmware selection.
- Loop-mount the rootfs from the `.wic`; confirm units, drop-ins, firmware and tools are present.
- Build packages individually first (`bitbake app-firmware`) to shorten the loop.

Rule: the build output is the product. No post-processing the image, no hand-repair on the board.

STEP 5 · BRING-UP

Verify one boundary at a time; fix in the sources

Boundary-by-boundary checks

```
xzcat app-image.wic.xz | sudo dd of=/dev/sdX \
     bs=4M conv=fsync status=progress
xmutil listapps                  # PL loaded
dmesg | grep -iE 'probe|error'   # drivers bound
i2cdetect -y N                   # peripherals
media-ctl -p ; aplay -l          # devices
modetest -c                      # display
journalctl -b -u app.service     # application
```

- Test controlled sources first, real inputs second, end-to-end last.
- Exercise lifecycle: cold boot, reboot, device unplug and return.

Working loop with the agent

- **Two terminals:** agent with project context in one; engineer executes in the other. Paste real output back, never a paraphrase.
- **Bounded checks:** the agent proposes the next smallest experiment and what each outcome would mean.
- **Diagnose live, fix at build time:** a manual drop-in on the board is a test of the hypothesis; the fix is a recipe change and a rebuild.
- **Demand mechanism:** when the agent swaps strings or settings, ask who consumes the value, where it is defined, and what it selects.
- **Let tools say no:** Vivado validation, DRC, timing, BitBake QA and the kernel are the authorities, not the draft.

Done when: a clean `bitbake` produces an image that boots on fresh media and works end-to-end, including recovery, with no manual steps.